CC1101 Guide And DoorGarageFob Deep Dive

Purpose Of This Document

This guide is the "study version" of the project documentation.

It is meant to explain:

1. how the CC1101 works in the specific asynchronous OOK mode used by this project,
2. how this firmware is structured from button press to RF burst,
3. what was wrong during bring-up and why each fix mattered,
4. how the recent dynamic-frequency and waveform changes fit together,
5. and which code snippet belongs to each major step of the flow.

The goal is not only to say what the code does. The goal is to make the design readable enough that you can come back later, study it calmly, and still understand why each piece exists.

Current Working Configuration

At the time of this document, the project is using:

- CC1101 in asynchronous OOK transmit mode
- software-generated waveform on the MCU side
- `GDO0` as the async data path
- 4 total frame emissions per button press
- `TX -> IDLE -> TX -> IDLE` cycling for repeated bursts
- runtime-selectable carrier frequency
- current target carrier set to `331 MHz`

The key constants are:

```
constexpr uint8_t  FRAME_TRANSMISSION_COUNT = 4;
constexpr uint32_t FREQ_331MHZ_BAND = 331000000;

constexpr uint8_t OOK_LOGIC_ZERO_POWER_BYTE = 0x00;
constexpr uint8_t OOK_LOGIC_ONE_POWER_INDEX = 1;
```

Those values live in `include/Config/Constants.h`.

What The CC1101 Is

The CC1101 is not just a raw RF output chip. It contains several subsystems that all matter during bring-up:

- an RF synthesizer that generates the carrier,
- a modem section that defines data rate, modulation, and channel bandwidth,
- a state machine that moves between `IDLE`, `TX`, `RX`, calibration, and sleep,
- an SPI register interface,
- GPIO-like `GDO` pins,
- and a PATABLE used for output-power selection.

In this project, we are not using the chip like a normal packet radio. We are using it more like a programmable RF modulator:

- the MCU creates the garage-door waveform in software,
- the CC1101 provides the carrier and OOK modulation behavior,
- and the async data path lets the waveform directly key the radio.

That is why tiny details like `GDO0` ownership, idle level, PATABLE setup, and `SIDLE` versus `STX` sequencing matter so much.

The Big Picture

At a high level, the signal path is:

1. the button press is debounced,
2. the callback enters a timing-critical transmit section,
3. the transceiver prepares the radio,
4. the frame streamer decides the order of preamble, bits, gaps, and repeats,
5. the encoder turns each symbol into timed `HIGH` / `LOW` pulses on the MCU pin connected to `GDO0`,
6. the CC1101 turns that async data into an RF OOK signal at the selected carrier frequency,
7. and the radio returns to `IDLE` between repeated bursts.

The architecture is split on purpose:

- `SPIBus` owns transport and timing of SPI transactions.
- `Transceiver` owns CC1101 state, configuration, frequency, PATABLE, and TX entry/exit.
- `SC41344_FrameStreamer` owns frame structure.
- `SC41344_Encoder` owns pulse timing.
- `main.cpp` owns application flow, watchdog handling, and button-driven orchestration.

Project Files That Matter Most

- `src/main.cpp`

- `src/SPI/SPIBus.cpp`
- `include/SPI/SPIBus.h`
- `src/Transciever/CC1101_Transceiver.cpp`
- `include/Transciever/CC1101_Transceiver.h`
- `include/Config/CC1101_Config/CC1101_LowBand_OOK_Config.h`
- `src/Encoder/SC41344_Encoder.cpp`
- `include/Streamer/SC41344_FrameStreamer.h`
- `include/utils/HelperConfigRegisters_CC1101.h`
- `docs/cc1101-stabilization.md`

The Current Startup Flow

This section walks through the actual startup path in the order it runs on the device.

Step 1: `setup()` Establishes The Safe MCU State

The project starts by disabling the watchdog, initializing the transceiver, and only then enabling the encoder:

```
wdt_disable();
LOG_NEW_LINE("Watchdog disabled during initialization");

transceiverReady = transceiver.begin();
if (!transceiverReady) {
    LOG_NEW_LINE("Transceiver initialization failed");
} else {
    LOG_NEW_LINE("Transceiver initialized successfully");
    encoder.begin();
    LOG_NEW_LINE("Encoder initialized");
}
```

This ordering matters because `encoder.begin()` configures the MCU pin that drives `GDO0`.

Earlier in the project, the encoder was initialized too early, which meant the MCU could take control of the same signal path that the radio was still trying to configure. That made bring-up less deterministic.

Step 2: `Transceiver::begin()` Controls The Whole Bring-Up Sequence

The core initialization flow is centralized in `Transceiver::begin()`:

```
bool Transceiver::begin()
{
    _spi.begin();

    if (!reset()) {
        LOG_NEW_LINE("Error: CC1101 reset sequence failed");
        return false;
    }

    if (!applyRegisterConfig_CC1101<RAMStoragePolicy>(
        Config_LowBand_OOK::setting_Regs.data(),
        Config_LowBand_OOK::setting_Regs.size(),
```

```
        writeLambda,
        readLambda
    )) {
        LOG_NEW_LINE("Error: Failed to apply CC1101 register configuration");
        return false;
    }

    setFrequency(_transceiver_config.getFrequencyHz());

    if (!configurePATable(_transceiver_config.getOOKLogicOnePowerByte())) {
        LOG_NEW_LINE("Error: Failed to configure CC1101 PATABLE");
        return false;
    }

    // Final PARTNUM / VERSION checks...
}
```

That one function is the center of the project because it defines the exact order of:

1. SPI setup,
2. reset,
3. base profile load,
4. dynamic frequency override,
5. PATABLE programming,
6. and final identity checks.

Step 3: The Manual Reset Sequence Must Follow The Datasheet

The reset code is more than just "send `SRES`".

The CC1101 requires correct line states and timing around `CSn` and `MISO`:

```
bool Transceiver::reset()
{
    pinMode(SCK, OUTPUT);
    pinMode(MOSI, OUTPUT);
    pinMode(MISO, INPUT);
    digitalWrite(SCK, HIGH);
    digitalWrite(MOSI, LOW);

    _spi.deselectDevice();
    delayMicroseconds(5);

    _spi.selectDevice();
    delayMicroseconds(10);

    _spi.deselectDevice();
    delayMicroseconds(41);

    _spi.selectDevice();

    while (digitalRead(MISO) == HIGH) {
        // wait until chip is ready
    }

    _spi.beginBus();
    _spi.transferRaw(static_cast<uint8_t>(CC1101::Strobes::Command::SRES));
    _spi.endBus();

    while (digitalRead(MISO) == HIGH) {
        // wait until reset is complete
    }
```

```
    _spi.deselectDevice();
    delay(10);

    return verifyChipId();
}
```

Why this matters:

- `CSn` timing is part of the CC1101 reset contract.
- `MISO` going low is the chip's "ready" signal.
- If the host starts clocking too early, the transaction can fail in a way that looks random.

That exact problem showed up during stabilization, so the project now treats "wait for ready" as a first-class requirement, not an optional delay.

Step 4: The SPI Layer Now Respects CC1101 Ready Timing

The project uses one shared SPI transaction wrapper:

```
template <typename Func>
inline bool SPIBus::applyTransaction(Func&& operation)
{
    beginBus();
    selectDevice();

    if (!waitUntilReady()) {
        deselectDevice();
        endBus();
        return false;
    }

    operation();
    deselectDevice();
    endBus();
    return true;
}
```

The important design choice here is that every normal transaction now waits for the CC1101 ready signal after `CSn` is asserted.

That fix solved one of the hardest bring-up problems: some reads and writes used to happen before the chip was ready, which created intermittent mismatches on otherwise-correct register writes.

Step 5: Read Validity No Longer Rejects Legitimate `0xFF`

One of the most misleading bugs in the earlier version was that a register value of `0xFF` was treated as an invalid read.

The current `ReadResult` logic is:

```
struct ReadResult {
    ReadResult(uint8_t status = 0xFF, uint8_t value = 0xFF)
        : status(status), value(value) {}

    bool isValid() const {
        return status != 0xFF;
    }
```

```
    uint8_t status;
    uint8_t value;
};
```

This matters because `PKTLEN = 0xFF` is a perfectly legal register value in this project.

Earlier, the firmware would read back `0xFF`, wrongly conclude the read failed, and then blame the SPI path. That bug created a false story about the radio health.

Step 6: The Register Table Is A Base Profile, Not The Final Carrier

The renamed file `include/Config/CC1101_Config/CC1101_LowBand_OOK_Config.h` is intentionally a base profile now.

That file still contains:

- OOK modem settings,
- low-band calibration defaults,
- packet engine choices for async mode,
- and startup default frequency bytes.

But the active carrier is no longer fixed by that file alone.

The project loads the base profile first, then overrides the frequency at runtime from `TransceiverConfig`.

Step 7: Register Configuration Is Policy-Driven

The helper that applies the table is responsible for write policy and verification policy:

```
template<typename StoragePolicy, typename Write, typename Read>
bool applyRegisterConfig_CC1101(
    const RegisterSettings* config,
    size_t N,
    Write&& writeRegister,
    Read&& readRegister)
{
    auto writeAndVerifyRegister = [&writeRegister, &readRegister](const RegisterSettings& regSettings) {
        uint8_t address = StoragePolicy::read(&regSettings.reg);
        uint8_t value = StoragePolicy::read(&regSettings.reg_value);
        bool verify = StoragePolicy::read(&regSettings.verify);

        if (!writeRegister(address, value)) {
            return false;
        }

        if (verify) {
            // retry readback up to 3 times
        }

        return true;
    };

    return avr_algorithms::for_each_until(config, N, writeAndVerifyRegister);
}
```

This separation is important:

- the SPI layer transports bytes,
- the config helper decides whether a register must be verified,
- and `SKIP_VERIFY` now genuinely means "write only, do not force readback."

That was another important fix during stabilization.

Step 8: Frequency Is Now Dynamic

The frequency setter computes the 24-bit CC1101 frequency word at runtime:

```
void Transceiver::setFrequency(uint32_t frequencyHz)
{
    constexpr uint32_t F_XOSC = 26000000;
    uint32_t freq = (uint64_t)frequencyHz * (1ULL << 16) / F_XOSC;

    writeRegister(CC1101::Address::FREQ2, (freq >> 16) & 0xFF);
    writeRegister(CC1101::Address::FREQ1, (freq >> 8) & 0xFF);
    writeRegister(CC1101::Address::FREQ0, freq & 0xFF);
}
```

This was one of the most important architectural improvements.

Before this change:

- the project had a frequency in `TransceiverConfig`,
- but the base register table also hardcoded the startup `FREQ2/FREQ1/FREQ0`,
- and the runtime configuration object was not actually winning.

Now the meaning is clear:

- the low-band profile gives the modem a sane starting point,
- and `TransceiverConfig` decides the real carrier.

For the current `331 MHz` target, the frequency word is:

- desired frequency: `331000000 Hz`
- frequency word: `0x0CBB13`
- bytes: `FREQ2 = 0x0C`, `FREQ1 = 0xBB`, `FREQ0 = 0x13`

That is why the project can now be retuned without creating a new whole config file for every carrier.

Step 9: PATABLE Now Models OOK Correctly

The current PATABLE logic is:

```
bool Transceiver::configurePATable(uint8_t powerLevelIndex)
{
    const uint8_t patable[8] = {
        OOK_LOGIC_ZERO_POWER_BYTE,
        powerLevelIndex,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00
```

```
    };

    if (!writeBurstRegister(CC1101::Address::PATABLE, patable, 8)) {
        return false;
    }

    powerLevelIndex = OOK_LOGIC_ONE_POWER_INDEX;

    uint8_t frend0 = readRegister(CC1101::Address::FREND0).value;
    frend0 &= ~0x07;
    frend0 |= (powerLevelIndex & 0x07);

    return writeRegister(CC1101::Address::FREND0, frend0);
}
```

This is a subtle but fundamental fix.

In OOK mode:

- logic `0` must correspond to carrier-off,
- logic `1` must correspond to carrier-on,
- and the selected PATABLE entry must match that design.

If `PATABLE[0]` is not `0x00`, then the "off" part of the waveform is not really off. That makes the signal look too filled-in or too continuous in URH.

Note:

The project still uses low-band PATABLE power values derived from the same family of low-band settings used around `315 MHz`. Since `331 MHz` is still in the same CC1101 low band, that is a reasonable starting point. It is still possible to fine-tune those values later if you want to optimize range, spectral cleanliness, or power behavior.

The Runtime Transmit Flow

After startup, the interesting part is what happens when the button is pressed.

Step 1: The Debounced Callback Creates A Timing-Critical Section

When a button press is confirmed, the callback does this:

```
void onButtonPressed()
{
    if (!transceiverReady) {
        return;
    }

    noInterrupts();
    wdt_disable();

    if (transceiver.transmitFrame(REMOTE1_OPEN_DOOR_CODE, encoder)) {
        LOG_NEW_LINE("Transmission successful");
    } else {
        LOG_NEW_LINE("Transmission failed");
    }

    wdt_enable(WDTO_8S);
    interrupts();
```

```
}
```

That code is simple, but the meaning is important:

- the critical timing of the RF waveform should not be interrupted by other ISRs,
- and the watchdog should not reset the MCU while the code is intentionally using blocking microsecond delays.

Step 2: Transmission Is Now Burst-Oriented

The transceiver no longer enters TX once for the whole repeated message. Instead it cycles TX and IDLE per emission:

```
template <size_t N>
inline bool Transceiver::transmitFrame(const uint8_t (&code_DataBits)[N], IBitEncoder& encoder)
{
    SC41344_FrameStreamer<N> streamer(*this);
    encoder.setIdle();

    for (uint8_t transmissionIndex = 0; transmissionIndex < FRAME_TRANSMISSION_COUNT; ++transmissionIndex) {
        if (transmissionIndex > 0) {
            encoder.sendSilence();
        }

        if (!enableTransmitMode()) {
            encoder.setIdle();
            return false;
        }

        streamer.streamFrameOnceStatic(code_DataBits, encoder, transmissionIndex == 0);

        if (!strobeCommand(CC1101::Strobes::Command::SIDLE)) {
            encoder.setIdle();
            return false;
        }
    }

    encoder.setIdle();
    return true;
}
```

This change solved a major waveform-matching problem.

Before the fix:

- the chip stayed in TX during the whole repeated sequence,
- gaps between repeats were not true radio-state boundaries,
- and the repeated bursts did not look like separate transmissions.

Now:

- each emission becomes a distinct RF burst,
- `SIDLE` is issued after each frame,
- and the repeated structure is much closer to what the original fob does.

Step 3: Entering TX Is Explicitly Verified

The helper that enters TX checks both the strobe result and the actual CC1101 state:

```
bool Transceiver::enableTransmitMode()
{
    using Strobe = CC1101::Strobes::Command;

    if (readRegister(CC1101::Address::MARCSTATE).value != 0x01) {
        if (!strobeCommand(Strobe::SIDLE)) {
            return false;
        }
        delayMicroseconds(10);
    }

    if (!strobeCommand(Strobe::STX)) {
        return false;
    }

    return readRegister(CC1101::Address::MARCSTATE).value == 0x13;
}
```

That is a good example of this project's design style:

- do not assume the chip entered the state you asked for,
- read back the state that matters,
- and fail early when the state machine does not match the expected mode.

Step 4: The Frame Streamer Owns Message Structure

The frame streamer decides what one repeated message looks like:

```
static void streamFrameOnceStatic(
    const uint8_t (&code_DataBits)[N],
    IBitEncoder& encoder,
    bool includePreamble = false)
{
    if (includePreamble) {
        encoder.sendPreamble();
    }

    avr_algorithms::for_each_element(code_DataBits, [&](uint8_t bit) {
        if (bit == 1) {
            encoder.sendOne();
        } else {
            encoder.sendZero();
        }
    });

    encoder.sendOpen();
}
```

And for the full repeated sequence:

```
for (uint8_t transmissionIndex = 0; transmissionIndex < FRAME_TRANSMISSION_COUNT; ++transmissionIndex) {
    if (transmissionIndex > 0) {
        encoder.sendSilence();
    }

    streamFrameOnceStatic(code_DataBits, encoder, transmissionIndex == 0);
}
```

This split is extremely useful conceptually:

- `Transceiver` owns radio state,
- `FrameStreamer` owns frame order,
- `Encoder` owns pulse shape.

That separation makes debugging much easier because you can ask "is the bug in state control, frame layout, or pulse timing?"

Step 5: The Encoder Owns Symbol Timing

The encoder converts logical symbols into timed waveform segments:

```
void SC41344_Encoder::sendOne()
{
    auto streamOneBitSeq = [&]()
    {
        _GDO0_pin.writePin(HIGH);
        delayMicroseconds(LONG_HIGH_US);
        _GDO0_pin.writePin(LOW);
        delayMicroseconds(SHORT_LOW_US);
    };

    avr_algorithms::repeat(2, streamOneBitSeq);
}

void SC41344_Encoder::sendZero()
{
    auto streamZeroBitSeq = [&]()
    {
        _GDO0_pin.writePin(HIGH);
        delayMicroseconds(SHORT_HIGH_US);
        _GDO0_pin.writePin(LOW);
        delayMicroseconds(LONG_LOW_US);
    };

    avr_algorithms::repeat(2, streamZeroBitSeq);
}
```

The supporting non-data states are just as important:

```
void SC41344_Encoder::sendSilence()
{
    _GDO0_pin.writePin(LOW);
    delayMicroseconds(FRAME_SILENCE_BETWEEN_WORDS);
}

void SC41344_Encoder::sendPreamble()
{
    _GDO0_pin.writePin(LOW);
    delayMicroseconds(PREAMBLE_LOW_DURATION_US);
}

void SC41344_Encoder::setIdle()
{
    _GDO0_pin.writePin(LOW);
}
```

The big lesson here is that idle level is not a cosmetic detail.

Holding the line `LOW` when idle prevents the CC1101 from seeing an unwanted "carrier on" condition when TX starts. That was one of the reasons the old signal looked wrong between repeated emissions.

Why The Design Is Split This Way

It can feel confusing at first that the project has both a transceiver, a frame streamer, and an encoder.

The clean mental model is:

- `Transceiver`: "Is the radio powered, reset, configured, and in the right state?"
- `FrameStreamer`: "What is the order of preamble, data bits, OPEN symbol, gaps, and repeats?"
- `Encoder`: "How does a logical symbol become an actual timed pulse sequence?"

If those responsibilities were collapsed into one class:

- the code would become harder to debug,
- RF-state bugs and waveform bugs would mix together,
- and future changes such as a different remote protocol would be harder to implement.

The Main Problems We Had And How Each One Was Fixed

This section is the heart of the debugging history.

1. The Register Table Size Was Wrong

Problem:

- `NUM_REG_TO_CONFIG_CC1101` did not match the actual number of entries in the table.
- The table had 23 real entries, but the size constant was larger.

Why that mattered:

- with `std::array`, the extra slot becomes zero-initialized,
- so the project could write one unintended "fake" entry during bring-up.

Fix:

- the constant was corrected to `23`.

Code anchor:

```
constexpr size_t NUM_REG_TO_CONFIG_CC1101 = 23;
```

2. The Encoder Claimed `GDO0` Too Early

Problem:

- `encoder.begin()` originally ran before the radio had completed bring-up.

Why that mattered:

- `GDO0` is central to the async transmit path,
- so configuring the MCU output too early could interfere with CC1101-side setup.

Fix:

- the encoder is now initialized only after `transceiver.begin()` succeeds.

3. `0xFF` Was Treated As An Invalid Register Value

Problem:

- the old read-validity logic rejected `value == 0xFF`.

Why that mattered:

- `PKTLEN = 0xFF` is normal in this project,
- so one correct register value created a false failure story.

Fix:

- read validity is now based on the status byte only.

4. Low-Level SPI Writes Were Verifying Even When The Policy Said Not To

Problem:

- `SPIBus::writeRegister()` used to do an immediate readback itself.

Why that mattered:

- it ignored the high-level `VERIFY` versus `SKIP_VERIFY` policy,
- and made certain register writes more fragile than intended.

Fix:

- verification now lives in `applyRegisterConfig_CC1101()`,
- and the SPI layer just transports bytes.

5. Normal SPI Transactions Did Not Wait For The Ready Signal

Problem:

- manual reset waited for `MISO` ready,

- but ordinary register reads and writes did not.

Why that mattered:

- some transactions started before the CC1101 was ready,
- which explains the intermittent readback mismatches seen earlier.

Fix:

- the shared SPI transaction wrapper now waits for ready every time.

6. Verification Was Too Brittle

Problem:

- a single bad readback could abort the full bring-up.

Why that mattered:

- during stabilization, some registers were correct on the next read,
- so the system failed too eagerly.

Fix:

- the verification helper now retries a few times before failing.

7. The Logs Were Too Hard To Read

Problem:

- earlier logs fragmented one event across many lines,
- and often omitted the register name or context.

Why that mattered:

- it made the root cause harder to see,
- especially when multiple problems were overlapping.

Fix:

- logging was rewritten into compact structured lines with register names, retries, expected values, and actual values.

8. Repeat Count Semantics Were Confusing

Problem:

- the old code structure made it easy to send "first frame + repeats" while reading the constant as "total frames."

Why that mattered:

- the code and intent drifted apart,
- and the number of on-air frames could be wrong.

Fix:

- the project now uses `FRAME_TRANSMISSION_COUNT`,
- which means total emissions per button press.

9. The Radio Stayed In TX Across The Whole Repeated Sequence

Problem:

- earlier, TX was entered once and held for the whole repeated message.

Why that mattered:

- the repeated bursts were not really separate radio bursts,
- which made the captured waveform structure look different from the fob.

Fix:

- the project now does `SIDLE / STX` around each repeated emission.

10. OOK Logic `0` Was Not Guaranteed To Be Truly Off

Problem:

- the old PATABLE setup did not model OOK low state cleanly.

Why that mattered:

- when logic `0` is not really carrier-off, URH sees a thicker or more filled-in signal than expected.

Fix:

- `PATABLE[0]` is now forced to `0x00`,
- and the high level is selected through `PATABLE[1]`.

11. Idle And Silence Behavior Could Produce Stray Carrier

Problem:

- idle and silence handling could leave the async line in a state that keyed carrier too early.

Why that mattered:

- even a tiny unintended carrier burst changes the visual pattern in the RF capture.

Fix:

- `sendSilence()` holds the line low for the whole gap,
- and `setIdle()` also holds the line low.

12. The Config Object Did Not Actually Own Frequency Selection

Problem:

- the base register table had fixed frequency bytes,
- and the transceiver config object was not overriding them afterward.

Why that mattered:

- changing the frequency in `main.cpp` did not guarantee the CC1101 really used it.

Fix:

- `Transceiver::begin()` now calls `setFrequency(_transceiver_config.getFrequencyHz())`.

13. The Profile File Name Became Misleading

Problem:

- once frequency became dynamic, the old name `CC1101_315MHZ_OOK_Config.h` suggested the project was still hard-fixed to 315 MHz.

Why that mattered:

- the codebase intent became harder to read.

Fix:

- the file was renamed to `CC1101_LowBand_OOK_Config.h`,
- and the namespace was renamed to `Config_LowBand_OOK`.

How The Current Signal Is Put Together

If we ignore the low-level SPI details for a moment, one full button-triggered RF transmission now looks like this:

1. hold the async line low,
2. enter TX,

3. send preamble on the first emission only,
4. send all data bits,
5. send the `OPEN` symbol,
6. return to `IDLE`,
7. hold the line low for the inter-frame gap,
8. repeat until 4 total emissions are complete.

That means there are two kinds of "silence" in the system:

- logic-level silence on the async data input,
- and RF-state silence because the CC1101 is returned to `IDLE`.

Both are required to make the waveform look right.

Why `331 MHz` Matters

One of the later discoveries was that the original fob appears to radiate around `331 MHz`, not `315 MHz`.

That difference is too large to treat as "close enough":

- `331 MHz - 315 MHz = 16 MHz`
- that is far outside the sort of fine offset a narrow OOK receiver would normally absorb

So the project was changed to make carrier selection dynamic and the active configuration in `main.cpp` was changed to `FREQ_331MHZ_BAND`.

The important architecture result is:

- the profile file is no longer the final authority on frequency,
- `TransceiverConfig` is.

Mental Model For Studying The Code

If you are reading this project later and want to understand it quickly, use this order:

1. `src/main.cpp`
2. `src/Transciever/CC1101_Transceiver.cpp`
3. `include/Transciever/CC1101_Transceiver.h`
4. `include/Config/CC1101_Config/CC1101_LowBand_OOK_Config.h`
5. `include/utils/HelperConfigRegisters_CC1101.h`
6. `include/SPI/SPIBus.h`
7. `src/SPI/SPIBus.cpp`
8. `include/Streamer/SC41344_FrameStreamer.h`

9. `src/Encoder/SC41344_Encoder.cpp`

That reading order follows the real execution flow.

Suggested Study Questions

When you revisit the code, these are good questions to ask:

1. Which class owns radio state versus waveform shape?
2. Why does the project use async OOK instead of the CC1101 TX FIFO?
3. Why is `PATABLE[0] = 0x00` so important?
4. Why is it safer to initialize the encoder after the radio?
5. Why does repeated transmission now cycle through `SIDLE` and `STX`?
6. What would break if the MCU left the async data line high while idle?
7. Why is dynamic frequency override cleaner than creating one config file per frequency?

If you can answer those comfortably, you understand the most important parts of this codebase.

Related Project Documents

- `docs/README.md`
- `docs/cc1101-stabilization.md`
- `docs/diagrams/runtime-sequence.md`
- `docs/diagrams/uml-class-diagram.md`

Final Summary

The project now works because several layers were aligned at the same time:

- the CC1101 reset path is reliable,
- SPI transactions honor the ready signal,
- the register profile is applied consistently,
- frequency is selected dynamically,
- OOK low state is truly carrier-off,
- TX is entered and exited per repeated frame,
- the frame count is explicit,
- and the encoder now drives `GDO0` in a way that matches the intended waveform.

That is the core idea to keep in mind:

The working solution did not come from one magic register. It came from making the transport layer, the state-machine layer, the frame-structure layer, and the pulse-shape layer all agree with each other.